

RECTIFICACIONES RÍO

Sistema de Gestión de Inventario

MANUAL TÉCNICO y MANUAL DE INSTALACIÓN

Índice de Contenidos

Índice de Contenidos	2
1. Descripción General del Sistema	4
2. Arquitectura del Sistema	4
2.1 Arquitectura General	4
2.2 Diagrama de Arquitectura	4
3. Lenguajes y Tecnologías Utilizadas	5
3.1 Backend	5
3.2 Frontend	5
3.3 Base de Datos	5
4. Estructura del Código	5
4.1 Organización de Archivos	5
4.2 Rutas Principales del Sistema	6
4.3 Decoradores de Seguridad	7
4.4 Módulo de Reportes PDF	7
5. Base de Datos	8
5.1 Motor y Plataforma	8
5.2 Tablas del Sistema	8
Tabla: usuarios	8
Tabla: categorias	8
Tabla: tipo	8
Tabla: piezas	9
Tabla: movimientos	9
5.3 Diagrama Entidad-Relación	9
5.4 Datos Iniciales	10
6. Diagramas UML y de Flujo	10
6.1 Diagrama de Flujo: Autenticación	10
6.2 Diagrama de Flujo: Registro de Movimiento	11
6.3 Diagrama de Casos de Uso	11
7. Requisitos de Hardware y Software	12
7.1 Requisitos Mínimos de Hardware	12
7.2 Software Requerido	12
8. Pasos de Instalación	12
8.1 Clonar el Repositorio	12
8.2 Crear y Activar un Entorno Virtual	13

En Windows:	13
En Linux / macOS:	13
8.3 Instalar las Dependencias	13
8.4 Configurar la Base de Datos.....	13
Opción A: Usar la base de datos en Railway (recomendada).....	13
Opción B: Configurar base de datos local	14
8.5 Ejecutar la Aplicación	14
9. Configuración Inicial del Sistema	15
9.1 Primer Inicio de Sesión.....	15
9.2 Panel Principal (Dashboard)	16
9.3 Gestión del Inventario.....	16
9.4 Registro de Movimientos	17
9.5 Generación de Reportes PDF.....	17
10. Solución de Problemas	18
11. Equipo de Desarrollo y Contacto.....	19

PARTE I

MANUAL TÉCNICO

1. Descripción General del Sistema

Rectificaciones Río es una aplicación web de gestión de inventario diseñada específicamente para talleres de rectificación de motores automotrices. Permite administrar el stock de piezas, registrar movimientos de entrada y salida, gestionar usuarios con distintos niveles de acceso y generar reportes en PDF de manera filtrada.

El sistema opera bajo un modelo cliente-servidor: el backend procesa la lógica de negocio y sirve las páginas HTML, mientras que una base de datos MySQL almacena toda la información de forma persistente.

2. Arquitectura del Sistema

2.1 Arquitectura General

El sistema sigue el patrón arquitectónico MVC (Modelo-Vista-Controlador), implementado sobre el framework Flask de Python:

- **Modelo:** Gestionado por MySQL a través del conector PyMySQL. Las tablas representan entidades del dominio (piezas, categorías, movimientos, usuarios).
- **Vista:** Plantillas HTML renderizadas por el motor Jinja2 de Flask, ubicadas en la carpeta /templates.
- **Controlador:** El archivo app.py contiene todas las rutas y la lógica de negocio del sistema.

2.2 Diagrama de Arquitectura

A continuación, se presenta el diagrama de capas del sistema:

CAPA DE PRESENTACIÓN (VISTA)		
login.html	dashboard.html	inventario / movimientos / usuarios.html
CAPA DE LÓGICA DE NEGOCIO (CONTROLADOR)		
app.py (Flask — Rutas, Decoradores, Lógica de negocio, Generación PDF)		
CAPA DE DATOS (MODELO)		

MySQL (Railway)	PyMySQL (conector)	config.py (configuración)
-----------------	--------------------	---------------------------

3. Lenguajes y Tecnologías Utilizadas

3.1 Backend

Tecnología	Uso en el Sistema
Python	Lenguaje principal del backend
Flask	Framework web, manejo de rutas y sesiones
PyMySQL	Conector para comunicación con MySQL
ReportLab	Generación de reportes en formato PDF
Cryptography	Soporte de cifrado para conexión segura
Jinja2	Motor de plantillas HTML dinámicas

3.2 Frontend

Tecnología	Uso en el Sistema
HTML5	Estructura de las páginas web
CSS3	Estilos visuales y diseño responsivo
JavaScript	Interactividad en el navegador
Bootstrap	Framework de diseño responsivo

3.3 Base de Datos

Tecnología	Uso en el Sistema
MySQL	Base de datos
Railway	Plataforma de hospedaje de la base de datos

4. Estructura del Código

4.1 Organización de Archivos

El proyecto está organizado de la siguiente manera:

Archivo / Carpeta	Descripción
app.py	Archivo principal. Contiene todas las rutas, lógica de negocio y generación de reportes PDF.
config.py	Configuración de la aplicación: SECRET_KEY y credenciales de la base de datos.
requirements.txt	Lista de dependencias Python necesarias para ejecutar el sistema.
inventario_taller_nuevo.sql	Script SQL para crear y poblar la base de datos con datos iniciales.
crear_funcion.py	Script auxiliar para creación de procedimientos o funciones en la BD.
importar.py	Script para importación masiva de datos a la base de datos.
templates/base.html	Plantilla base con navegación, cabecera y pie de página. Todas las demás heredan de ésta.
templates/login.html	Formulario de autenticación de usuarios.
templates/dashboard.html	Panel principal con estadísticas, alertas y últimos movimientos.
templates/inventario.html	Gestión del catálogo de piezas: CRUD completo, filtros y categorías.
templates/movimientos.html	Registro de entradas y salidas de piezas del inventario.
templates/usuarios.html	Administración de usuarios (solo para el rol Administrador).

4.2 Rutas Principales del Sistema

A continuación, se detallan las rutas (endpoints) definidas en app.py y su función:

Ruta	Método	Acceso	Descripción
/login	GET/POST	Público	Formulario de inicio de sesión
/logout	GET	Autenticado	Cierra la sesión del usuario
/dashboard	GET	Autenticado	Panel con estadísticas y alertas de stock
/inventario	GET	Autenticado	Lista y gestión de piezas
/inventario/agregar	POST	Autenticado	Agrega una nueva pieza al inventario
/inventario/editar/<id>	POST	Autenticado	Actualiza datos de una pieza existente
/inventario/eliminar/<id>	POST	Admin	Elimina una pieza y sus movimientos
/movimientos	GET	Autenticado	Lista de todos los movimientos

Ruta	Método	Acceso	Descripción
/movimientos/registrar	POST	Autenticado	Registra entrada o salida de piezas
/usuarios	GET	Admin	Gestión de usuarios del sistema
/usuarios/agregar	POST	Admin	Crea un nuevo usuario
/usuarios/eliminar/<id>	POST	Admin	Elimina un usuario del sistema
/reporte/inventario	GET	Autenticado	Genera y descarga un reporte PDF filtrado
/api/categorias	GET	Autenticado	API JSON con la lista de categorías

4.3 Decoradores de Seguridad

El sistema implementa dos decoradores personalizados para proteger las rutas:

- **@login_required:** Verifica que exista una sesión activa. Si no hay usuario en sesión, redirige a /login. Se aplica a todas las rutas que requieren autenticación.
- **@admin_required:** Además de verificar la sesión, comprueba que el rol sea 'Administrador'. Si no lo es, redirige al dashboard con mensaje de error. Se aplica a rutas de administración de usuarios, eliminación de piezas y gestión de tipos/categorías.

4.4 Módulo de Reportes PDF

El sistema genera reportes en formato PDF usando la biblioteca ReportLab. El módulo de reporte (/reporte/inventario) soporta múltiples filtros combinables:

- Rango de fechas (fecha_inicio / fecha_fin)
- Nombre de pieza (búsqueda parcial con LIKE)
- Categoría (id_categoria)
- Tipo de movimiento (ENTRADA / SALIDA)
- Estado del stock (bajo / normal)

El PDF generado incluye: encabezado de la empresa, resumen de filtros aplicados, tabla de inventario de piezas y tabla de movimientos. El diseño utiliza colores rojo oscuro (#8B0000) para los encabezados, coherente con la identidad visual del sistema.

5. Base de Datos

5.1 Motor y Plataforma

El sistema utiliza MySQL 8.0.46 como motor de base de datos, alojado en Railway (plataforma cloud). La conexión se establece mediante PyMySQL con los parámetros definidos en config.py.

5.2 Tablas del Sistema

La base de datos inventario_taller contiene las siguientes tablas:

Tabla: usuarios

Almacena las credenciales y roles de los usuarios del sistema.

Campo	Tipo	Restricciones	Descripción
id_usuario	INT	PK, AUTO_INCREMENT	Identificador único del usuario
usuario	VARCHAR(30)	—	Nombre de usuario para inicio de sesión
contrasena	VARCHAR(30)	—	Contraseña del usuario
rol	VARCHAR(20)	—	Rol del usuario: 'Administrador' o 'Empleado'

Tabla: categorias

Clasifica las piezas del inventario en grupos temáticos.

Campo	Tipo	Restricciones	Descripción
id_categoria	INT	PK, AUTO_INCREMENT	Identificador único de la categoría
nombre_categoria	VARCHAR(50)	—	Nombre descriptivo: Motor, Bielas, Pistones, etc.

Tabla: tipo

Clasifica las piezas según el tipo de sistema de combustible del motor.

Campo	Tipo	Restricciones	Descripción
id_tipo	INT	PK, AUTO_INCREMENT	Identificador único del tipo

Campo	Tipo	Restricciones	Descripción
nombre_tipo	VARCHAR(50)	—	Tipo de motor: Carburado, TBI o FI (inyección)

Tabla: piezas

Tabla central del inventario. Registra cada pieza con sus atributos y stock actual.

Campo	Tipo	Restricciones	Descripción
id_pieza	INT	PK, AUTO_INCREMENT	Identificador único de la pieza
nombre_pieza	VARCHAR(50)	—	Nombre descriptivo de la pieza
año	INT	—	Año del modelo compatible con la pieza
cantidad	INT	DEFAULT 0	Cantidad actual en stock
descripcion	VARCHAR(100)	—	Descripción adicional de la pieza
id_categoria	INT	FK → categorias	Categoría a la que pertenece la pieza
id_tipo	INT	FK → tipo	Tipo de motor compatible

Tabla: movimientos

Registra cada entrada o salida de piezas, funcionando como bitácora de trazabilidad.

Campo	Tipo	Restricciones	Descripción
id_movimiento	INT	PK, AUTO_INCREMENT	Identificador único del movimiento
id_pieza	INT	FK → piezas	Pieza involucrada en el movimiento
tipo_movimiento	VARCHAR(20)	—	Tipo: ENTRADA o SALIDA
cantidad	INT	—	Cantidad de unidades movidas
fecha	DATETIME	—	Fecha y hora exacta del movimiento
proveedor	VARCHAR(100)	—	Proveedor o destino del movimiento

5.3 Diagrama Entidad-Relación

A continuación, se muestra el diagrama ER de la base de datos:

categorias PK id_categoria nombre_categoria	piezas (CENTRAL) PK id_pieza nombre_pieza, año cantidad, descripcion FK id_categoria FK id_tipo	tipo PK id_tipo nombre_tipo
1 ——— N	↕ 1 a N ↕	1 ——— N
movimientos PK id_movimiento FK id_pieza tipo_movimiento cantidad fecha proveedor		
usuarios (independiente) PK id_usuario usuario contrasena rol		

5.4 Datos Iniciales

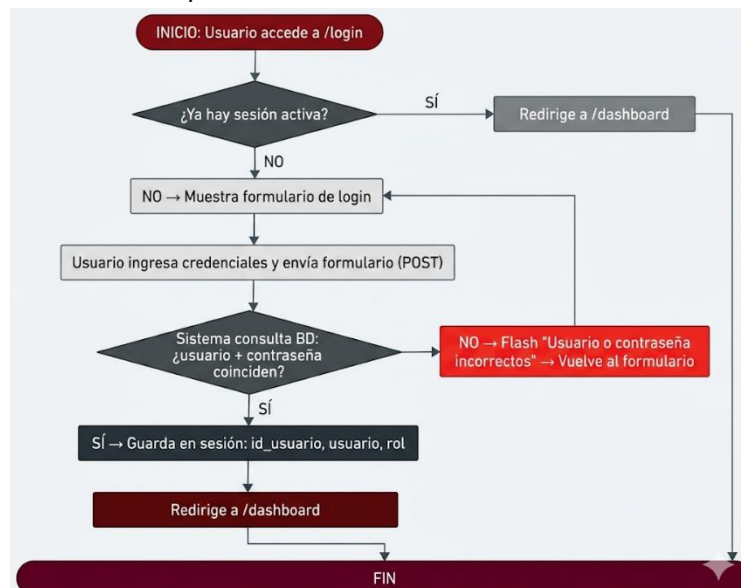
El script SQL incluye datos de prueba para las tablas maestras:

- **Categorías (13):** Motor, Monoblocks, Cabezas, Bielas, Pistones, Cigüeñales, Válvulas, Árbol de levas, Bomba de aceite, Bomba de agua, Metales de biela, Metales de centro, Kit de distribución.
- **Tipos (3):** Carburado, TBI (Throttle Body Injection), FI (Fuel Injection completo).
- **Usuarios (2):** Miguel: admin123 (Administrador). empleado1:1234 (Empleado).

6. Diagramas UML y de Flujo

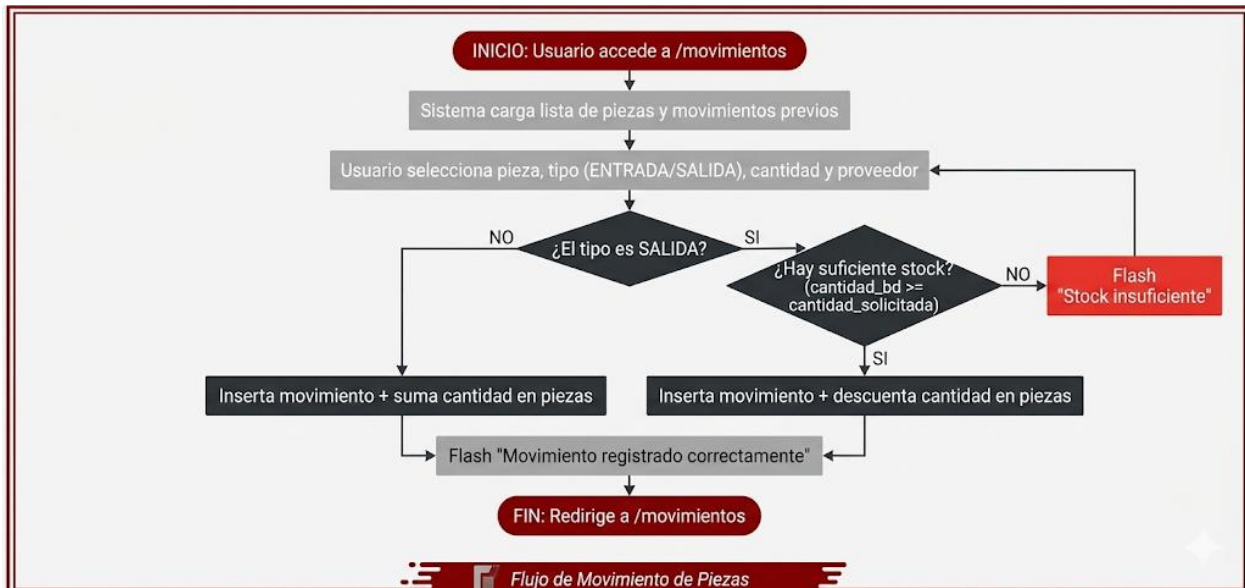
6.1 Diagrama de Flujo: Autenticación

El siguiente diagrama muestra el proceso de inicio de sesión del sistema:



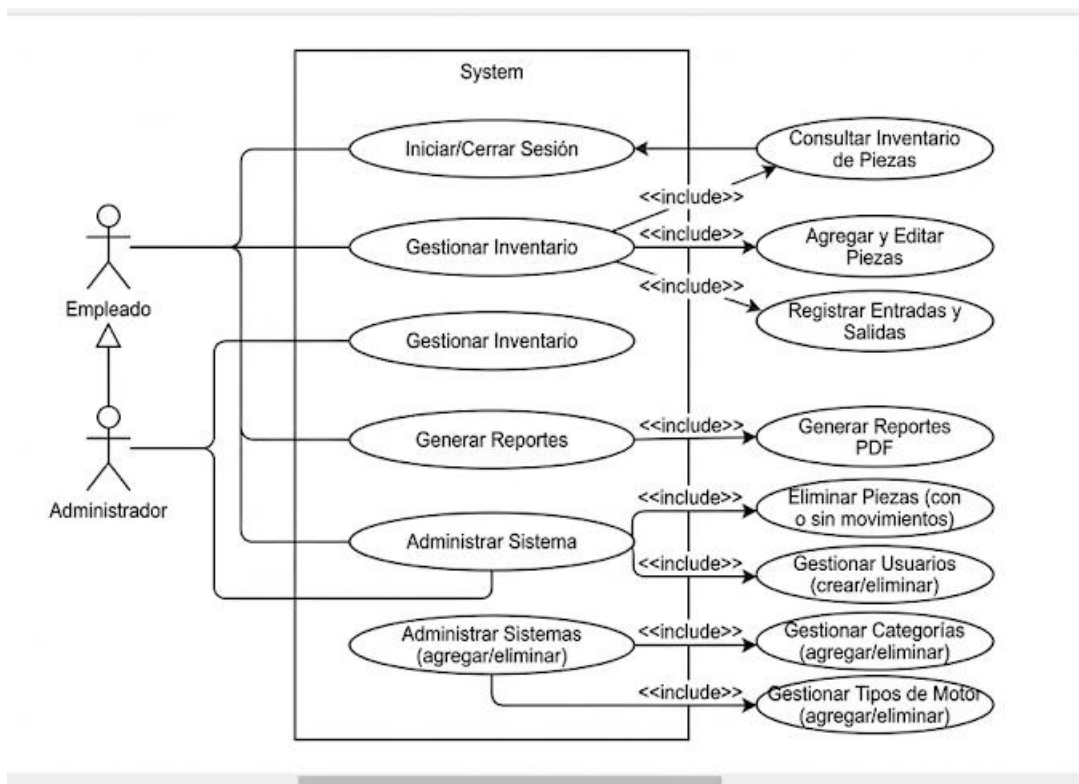
6.2 Diagrama de Flujo: Registro de Movimiento

Proceso para registrar una entrada o salida de piezas del inventario:



6.3 Diagrama de Casos de Uso

Actores y funcionalidades del sistema según el rol:



PARTE II

MANUAL DE INSTALACIÓN

7. Requisitos de Hardware y Software

7.1 Requisitos Mínimos de Hardware

Componente	Mínimo	Recomendado
Conexión a Internet	Requerida (BD en Railway)	Banda ancha estable
Sistema Operativo	Windows 10 / Linux	Windows 11 / Linux

7.2 Software Requerido

Software	Descripción
Python	Intérprete del lenguaje. Disponible en python.org
pip	Gestor de paquetes de Python (incluido con Python)
MySQL Server	Motor de base de datos (o acceso a Railway)
Git	Para clonar el repositorio del proyecto
Navegador web moderno	Chrome, Firefox, Edge o Safari actualizados

8. Pasos de Instalación

8.1 Clonar el Repositorio

Abre una terminal y ejecuta el siguiente comando para obtener el código fuente:

```
C:\Users\likeb>git clone https://github.com/EvilEllie/Proyecto-Taller.git
```

8.2 Crear y Activar un Entorno Virtual

Se recomienda usar un entorno virtual para aislar las dependencias del proyecto:

En Windows:

```
python -m venv venv  
venv\Scripts\activate
```

En Linux / macOS:

```
python3 -m venv venv  
source venv/bin/activate
```

Una vez activado, el prompt de la terminal mostrará (venv) al inicio, indicando que el entorno virtual está activo.

```
C:\Users\likeb\OneDrive\Escritorio\ejemplos\Proyecto-Taller>python -m venv venv  
C:\Users\likeb\OneDrive\Escritorio\ejemplos\Proyecto-Taller>venv\Scripts\activate  
(venv) C:\Users\likeb\OneDrive\Escritorio\ejemplos\Proyecto-Taller>pip install requirements.txt
```

8.3 Instalar las Dependencias

Con el entorno virtual activo, instala todas las bibliotecas necesarias ejecutando:

```
pip install -r requirements.txt
```

Este comando instalará automáticamente: Flask 3.1.0, PyMySQL 1.1.1, ReportLab 4.4.0 y Cryptography 44.0.2.

```
(venv) C:\Users\likeb\OneDrive\Escritorio\ejemplos\Proyecto-Taller>pip install -r requirements.txt  
Collecting Flask==3.1.0 (from -r requirements.txt (line 1))  
  Downloading flask-3.1.0-py3-none-any.whl.metadata (2.7 kB)  
Collecting pymysql==1.1.1 (from -r requirements.txt (line 2))  
  Downloading PyMySQL-1.1.1-py3-none-any.whl.metadata (4.4 kB)  
Collecting cryptography==44.0.2 (from -r requirements.txt (line 3))  
  Downloading cryptography-44.0.2-cp39-abi3-win_amd64.whl.metadata (5.7 kB)  
Collecting reportlab==4.4.0 (from -r requirements.txt (line 4))
```

8.4 Configurar la Base de Datos

Opción A: Usar la base de datos en Railway (recomendada)

El sistema ya está configurado para conectarse a la base de datos en Railway. Los parámetros en config.py son:

```
MYSQL_HOST = 'mysql.railway.internal'
```

```
MYSQL_USER = 'root'
MYSQL_DB = 'railway'
MYSQL_PORT = 3306
```

No se requiere ninguna configuración adicional si se usa Railway.

Opción B: Configurar base de datos local

Si se desea usar una instancia local de MySQL, sigue estos pasos:

1. Abre MySQL Workbench o la consola de MySQL.
2. Importa el archivo inventario_taller_nuevo.sql:

```
mysql -u root -p < inventario_taller_nuevo.sql
```

3. Edita config.py con tus credenciales locales:

```
class Config:
    SECRET_KEY = 'inventario_taller_secretkey_2026'

    MYSQL_HOST = 'localhost'
    MYSQL_USER = 'root'
    MYSQL_PASSWORD = '2005'
    MYSQL_DB = 'railway'
    MYSQL_PORT = 3306
```

8.5 Ejecutar la Aplicación

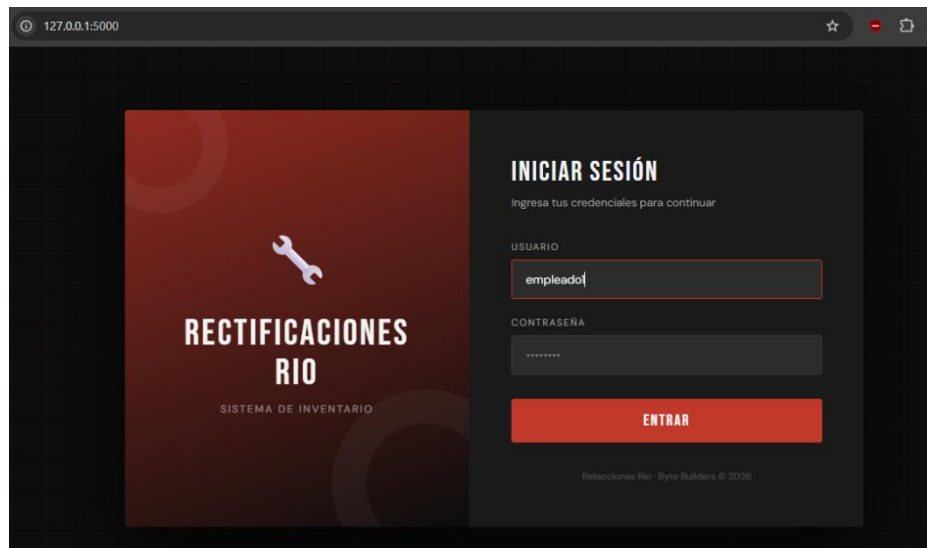
Con la base de datos configurada, inicia el servidor Flask ejecutando:

```
python app.py
```

Si todo está correcto, verás en la terminal un mensaje similar a:

```
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Abre tu navegador y accede a la dirección: `http://127.0.0.1:5000`

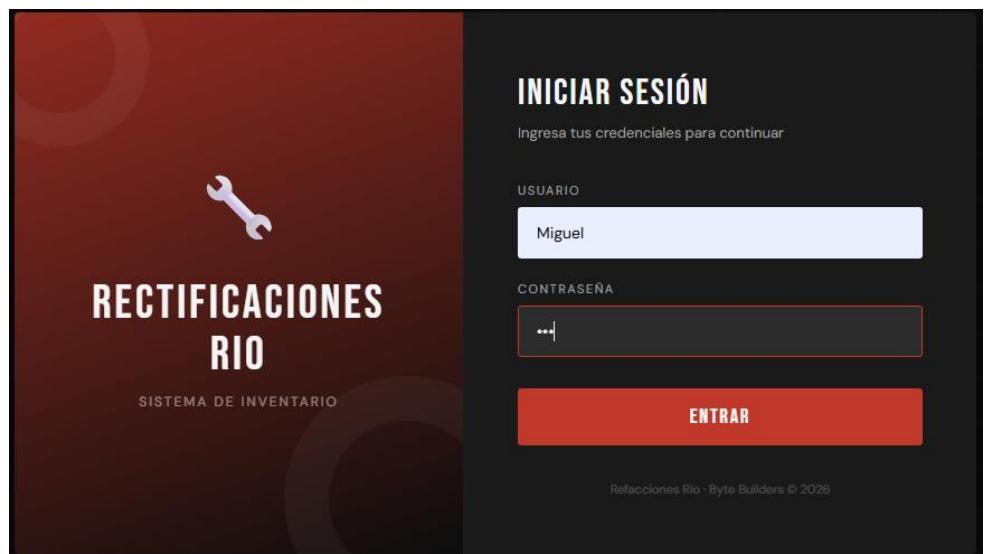


9. Configuración Inicial del Sistema

9.1 Primer Inicio de Sesión

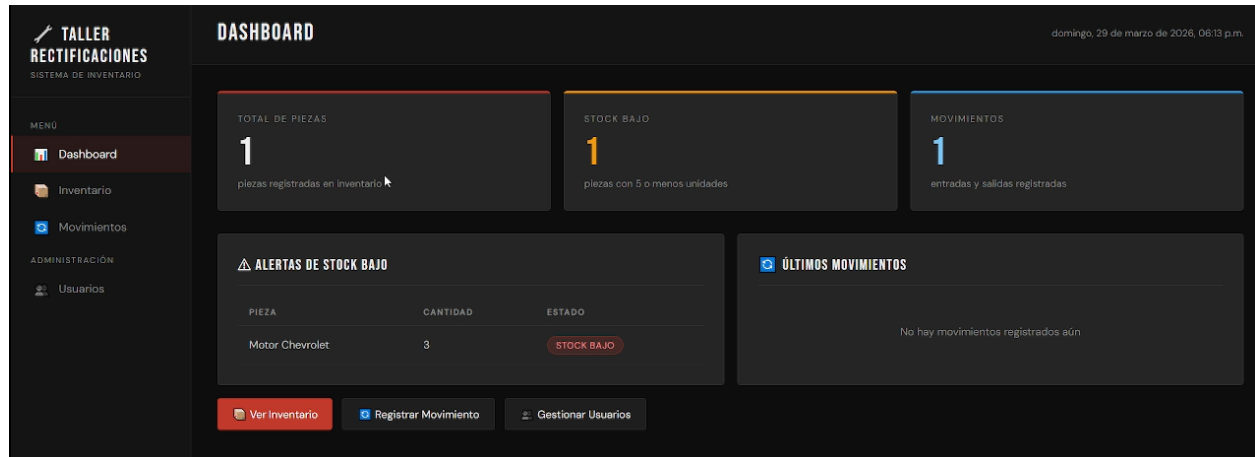
Al acceder por primera vez a `http://localhost:5000`, el sistema mostrará la pantalla de inicio de sesión. Usa las credenciales de administrador incluidas en los datos de prueba:

Campo	Valor
Usuario	Miguel
Contraseña	admin123
Rol	Administrador



9.2 Panel Principal

Tras iniciar sesión, el sistema mostrará con:



9.3 Gestión del Inventario

En la sección de Inventario el administrador puede agregar nuevas piezas especificando nombre, año, cantidad, descripción, categoría y tipo. El sistema valida que no existan duplicados (mismo nombre y año).

+ AGREGAR NUEVA PIEZA

NOMBRE DE LA PIEZA

Motor Toyota

AÑO

2020

CANTIDAD INICIAL

8

CATEGORÍA

Motor

TIPO

Carburado

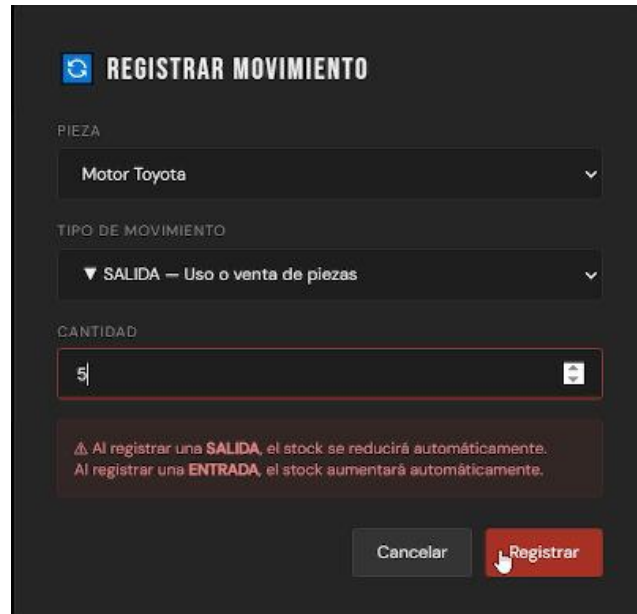
DESCRIPCIÓN

Pieza de ejemplo numero 2

Cancelar Guardar Pieza

9.4 Registro de Movimientos

En la sección de Movimientos se registran las entradas y salidas de piezas. El sistema valida automáticamente que haya stock suficiente antes de permitir una salida.



El formulario 'REGISTRAR MOVIMIENTO' tiene un fondo oscuro. En la parte superior, a la izquierda, hay un icono de flecha circular azul, seguido del título 'REGISTRAR MOVIMIENTO' en letras blancas y mayúsculas. Debajo del título, hay tres campos de entrada: 1. 'PIEZA' con un menú desplegable que muestra 'Motor Toyota' y una flecha hacia abajo. 2. 'TIPO DE MOVIMIENTO' con un menú desplegable que muestra '▼ SALIDA — Uso o venta de piezas' y una flecha hacia abajo. 3. 'CANTIDAD' con un campo de texto que contiene el número '5' y un icono de borrar a la derecha. Debajo de estos campos, hay un mensaje de advertencia en un recuadro rojo oscuro con texto blanco: '⚠ Al registrar una **SALIDA**, el stock se reducirá automáticamente. Al registrar una **ENTRADA**, el stock aumentará automáticamente.' En la parte inferior derecha, hay dos botones: 'Cancelar' (gris) y 'Registrar' (rojo).

9.5 Generación de Reportes PDF

El botón 'Generar Reporte' en el dashboard abre un modal donde se pueden aplicar filtros (fecha, categoría, tipo de movimiento, estado de stock) antes de generar el PDF. El reporte se descarga automáticamente en el navegador.

10. Solución de Problemas

A continuación, se describen los errores más comunes y su solución:

Error / Síntoma	Causa Probable	Solución
ModuleNotFoundError: No module named 'flask'	Las dependencias no están instaladas o el entorno virtual no está activo	Activar el entorno virtual y ejecutar: pip install -r requirements.txt
pymysql.err.OperationalError: (2003) Can't connect to MySQL	La base de datos no está accesible (sin internet o credenciales incorrectas)	Verificar conexión a internet para Railway, o revisar config.py para BD local
Address already in use (port 5000)	Otro proceso está usando el puerto 5000	Cerrar el proceso conflictivo o cambiar el puerto en app.run(port=5001)
Error 403 / 'No tienes permisos'	El usuario con rol Empleado intenta acceder a una ruta de Administrador	Iniciar sesión con una cuenta de Administrador para acceder a esas funciones
Flash: 'Stock insuficiente para registrar la salida'	Se intenta sacar más unidades de las disponibles en el inventario	Registrar una ENTRADA previa para reponer el stock antes de registrar la salida
Flash: 'Ya existe una pieza con ese nombre y año'	Intento de duplicar una pieza existente	Verificar el inventario antes de agregar. Editar la pieza existente si es necesario
No se encontraron datos con los filtros seleccionados	Los filtros del reporte no coinciden con ningún registro	Ampliar el rango de fechas o reducir los filtros aplicados

11. Equipo de Desarrollo y Contacto

Para soporte técnico, mantenimiento o cualquier duda relacionada con el sistema, contactar a cualquiera de los integrantes del equipo de desarrollo:

Nombre	Teléfono
Yuliana Ferretiz Meza	487 102 5354
Perla Viviana Toledo López	444 122 5065
Sandra Paola Torres Carbajal	487 113 0876
Jorge Loredó Rojas	487 107 1712
Jesús Tadeo Varela Martínez	487 166 2558
Joel Fernando Juárez Barrón	487 117 0355